Università degli Studi di Salerno
Dipartimento di Informatica
Corso di Fondamenti di Intelligenza Artificiale

SyntheticSoul

Deep Reinforcement Learning Agents for the Mantis Lords Boss Fight in Hollow Knight

Students:

Lamberti Giovanni
mat. 0512118718
Plumitallo Antonio
mat. 0512119043
Senatore Francesco
mat. 0512120568
Ricciardelli Domenico
mat. 0512119238

Professor:

Prof. Fabio Palomba

Academic Year 2025/2026

GitHub Repository: https://github.com/jlgionny/SyntheticSoul

Contents

1 Introduction and Problem Definition **3**
1.1 Introduction . . . 3
1.2 PEAS Specification . . . 3
1.3 Environment Characteristics . . . 4
1.4 Technical Challenges . . . 4

2 System Architecture **5**
2.1 Architectural Overview . . . 5
2.2 Communication Protocol . . . 5
2.3 Game State Extraction (GameStateExtractor) . . . 6
2.4 Attack Pattern Detection (MantisAttackDetector) . . . 7
2.5 Action Execution (ActionExecutor) . . . 7
2.6 Victory Management and Episode Lifecycle . . . 8

3 State Preprocessing **9**
3.1 Version V1 — 34 Features . . . 9
3.2 Version V2 — 51 Features . . . 9
3.3 Frame Stacking . . . 9

4 AI Solution: Deep Reinforcement Learning **11**
4.1 Action Space . . . 11
4.2 Experience Management (Dynamic Dataset) . . . 11
4.2.1 Transition Structure . . . 11
4.2.2 Replay Buffer (DQN) . . . 11
4.3 Deep Q-Network (DQN) . . . 12
4.3.1 Architecture: Dueling DQN . . . 12
4.3.2 DQN Training Mechanisms . . . 14
4.4 Proximal Policy Optimization (PPO) . . . 15
4.4.1 Actor-Critic Architecture and Frame Stacking . . . 15
4.4.2 Clipped Objective Function . . . 15
4.4.3 Advanced PPO Mechanisms . . . 16
4.5 DQN vs PPO Comparison . . . 16
4.5.1 Architectural and Memory Comparison . . . 17
4.5.2 Complexity Analysis . . . 17
4.5.3 Training Performance Evaluation . . . 18

5 Experiments: 4-Phase Curriculum Learning **19**
5.1 Phase Pipeline . . . 19
5.2 Reward Function Details . . . 19
5.3 Pattern Reward Bonus . . . 20

6 Multi-Instance Training **21**

6.1 Distributed Training Architecture 21
6.2 Hall of Fame 21
6.3 Champion System 21
6.4 Per-Phase Hyperparameter Configuration 22

7 Implementation Details **23**
7.1 Numerical Stability 23
7.2 Technology Stack 23
7.3 Project Structure 23

8 Conclusions and Future Work **25**
8.1 Main Contributions 25
8.2 Future Work 25

Chapter 1

Introduction and Problem Definition

1.1 Introduction

The **SyntheticSoul** project explores the application of Deep Reinforcement Learning (DRL) in complex, non-deterministic game environments. Unlike classical AI domains such as chess or Go, where the state space is discrete and fully observable, *Hollow Knight* features a continuous state space, real-time dynamics and adversaries with stochastic attack patterns.

Hollow Knight is a 2D action-platformer developed by Team Cherry, renowned for its precise and challenging combat system. The **Mantis Lords** represent one of the game's most iconic boss fights: a multi-phase encounter against agile enemies that attack with horizontal dashes, diagonal lunges, boomerang projectile throws and vertical plunges. The second phase introduces *two simultaneous mantises*, demanding heightened spatial awareness and the ability to manage multiple threats concurrently.

The main goal of the project is to train autonomous agents capable of defeating the Mantis Lords in the **Hall of Gods** (the game's boss-rush mode), comparing the performance of two fundamental Deep RL approaches:

1. **DQN** (Deep Q-Network) — a *value-based, off-policy* method that estimates the value of each action through a Dueling neural network;
2. **PPO** (Proximal Policy Optimization) — a *policy-based, on-policy* method that directly learns a stochastic policy, augmented with **Frame Stacking** to capture temporal dynamics.

1.2 PEAS Specification

The agent's operating environment is defined according to the PEAS framework:

- **Performance:** Survival time (episode duration), damage dealt to the boss, number of mantises killed, final victory (all three mantises defeated).
- **Environment:** Mantis Lords arena in the Hall of Gods (Godhome) — a rectangular platform with lateral spikes and a multi-phase boss.
- **Actuators:** Virtual controller with 8 discrete actions: movement (left, right, up, down), jump, attack, dash and spell.
- **Sensors:** Direct extraction from game memory (HP, position, velocity of player and boss), terrain raycasts, PlayMaker FSM readings for attack patterns, projectile detection.

1.3 Environment Characteristics

- **Single agent:** only one agent (the knight) must defeat the boss.
- **Partially observable:** the agent perceives only the information extracted by the mod, not the entire visual scene.
- **Stochastic:** the boss selects attack patterns pseudo-randomly via PlayMaker FSMs.
- **Dynamic:** the game advances in real time regardless of the agent's decisions.
- **Continuous:** positions, velocities and distances are floating-point values.
- **Episodic:** each attempt (from scene load to death or victory) constitutes an independent episode.

1.4 Technical Challenges

The project tackles several non-trivial challenges:

- **Game interfacing:** Hollow Knight exposes no API for external agents; a complete Unity/C# mod had to be developed.
- **Real-time synchronization:** the game–agent loop must operate at 20 Hz without desynchronization.
- **Heterogeneous observation space:** the state must be extracted from diverse Unity components (Rigidbody2D, PlayMaker FSM, HealthManager, etc.).
- **Multi-phase boss:** the Mantis Lords change behaviour after the first kill, switching from a single opponent to two simultaneous attackers.
- **Reward shaping:** designing reward functions that progressively guide learning without introducing degenerate behaviours.

Chapter 2

System Architecture

The SyntheticSoul system comprises two macro-components: a **C# mod** running inside the Hollow Knight process (Unity side) and a **Python training framework** executing the RL algorithms. Communication takes place over TCP sockets on localhost.

2.1 Architectural Overview

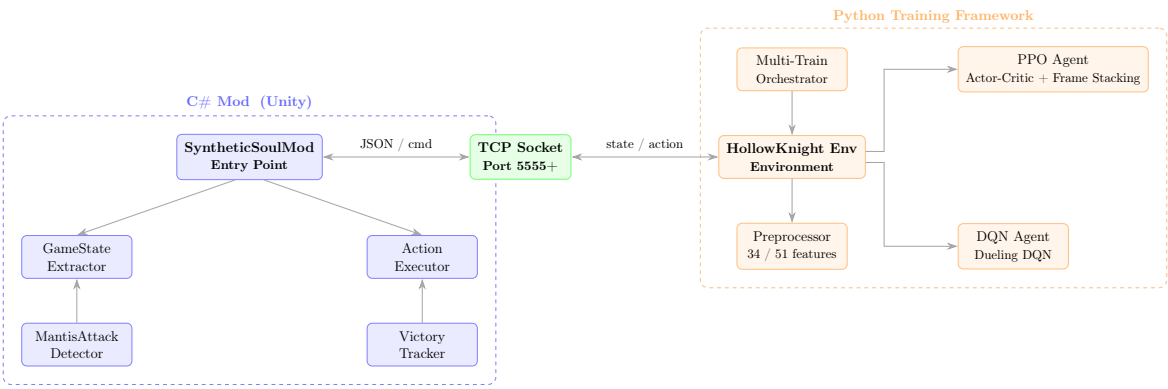

Figure 2.1: System architecture: communication between the C# Mod and the Python Training Framework via TCP sockets.

2.2 Communication Protocol

Communication between the mod and the agent follows a synchronous *request–response* protocol:

1. The mod extracts the game state and serializes it as **JSON**.
2. The JSON string is sent to the Python client via TCP socket (newline-delimited).
3. Python receives the state, preprocesses it, selects an action and sends back a command string (e.g. `MOVE_LEFT`, `ATTACK`, `DASH`).
4. The mod receives the command and executes it through the virtual input device.

This loop operates at approximately **20 Hz** (`UPDATE_INTERVAL = 0.05 s`). Socket reads on the mod side are **blocking**: the game waits for Python's response before proceeding, ensuring perfect synchronization at the cost of inference speed. The `SocketCommunicator` component manages persistent connections with automatic reconnection, command queues for burst handling and configurable timeouts (10 s).

```
// The read BLOCKS the thread until Python responds.
// This forces the game to run at Python's speed.
byte[] buffer = new byte[4096];
int bytesRead = stream.Read(buffer, 0, buffer.Length);

if (bytesRead > 0)
{
    string rawData = Encoding.UTF8.GetString(buffer, 0, bytesRead);
    string[] commands = rawData.Split(
        new char[] { '\n', '\r' },
        StringSplitOptions.RemoveEmptyEntries);
    foreach (string cmd in commands)
        commandQueue.Enqueue(cmd.Trim().ToUpper());
}
```

Listing 2.1: Blocking socket read (SocketCommunicator.cs)

2.3 Game State Extraction (GameStateExtractor)

The class `GameStateExtractor.cs` extracts the game state at every step. The `GameState` structure contains roughly 80 fields organized into categories:

Table 2.1: Game state categories extracted by the mod.

Category	**Key Fields**
Player	Position (x, y), velocity (v_x, v_y), HP, Soul, isGrounded, canDash, canAttack
Boss (threat-locked)	Position, velocity, HP, relative distance and position to player, isFacingBoss
Boss Intent	Current action with priority: Drop > Dash > Throw > Idle
Hazards	Nearby projectiles with relative position, velocity, distance
Attack Patterns	Primary/secondary mantis pattern (8 patterns + wind-up/active/recovery phases)
Terrain	5 raycast values detecting ground below and beside the player
Victory Tracker	Session wins, current streak, win rate

A critical aspect is the **threat locking** mechanism: boss-related fields always point to the most dangerous mantis — the one in an active attack phase closest to the player — rather than simply the nearest one by distance.

```
private void ExtractTerrainInfo(HeroController hero, GameState
    state)
{
    Vector2 rayOrigin = new Vector2(playerPos.x, playerPos.y + 0.5f
    );
    // Downward raycast
    RaycastHit2D groundHit = Physics2D.Raycast(
        rayOrigin, Vector2.down, 5f, terrainLayer);
    // Normalization (0-1)
    state.terrainInfo[0] = (groundHit.collider != null)
```

```
        ? Mathf.Clamp01(groundHit.distance / 3f) : 1.0f;
}
```

Listing 2.2: Terrain extraction via Raycast (GameStateExtractor.cs)

2.4 Attack Pattern Detection (MantisAttackDetector)

The attack-pattern detector is one of the project's most significant contributions. It combines three information sources to classify each mantis's behaviour in real time:

1. **PlayMaker FSM reading:** each mantis has an FSM named "Mantis Lord" whose states correspond to different attacks. The detector reads the active state and classifies it via substring matching.
2. **Kinematic analysis:** the mantis's velocity (from Rigidbody2D or derived from position deltas) confirms or disambiguates the pattern.
3. **Temporal tracking:** the duration of each state is tracked to identify wind-up, active attack and recovery phases.

Table 2.2: Mantis Lords attack patterns recognized by the detector.

ID	Pattern	Description & Kinematic Thresholds
0	IDLE	No attack in progress
1	DASH_HORIZ	Horizontal ground lunge ($\lvert v_x \rvert > 5$, $\lvert v_y \rvert < 3$)
2	DASH_DIAG	Diagonal dash from above ($\lvert v_x \rvert > 3$ and $\lvert v_y \rvert > 3$)
3	DAGGER_THROW	Boomerang sickle throw from wall
4	WALL_THROW	Attack from lateral wall
5	LAND_ATTACK	Vertical plunge from above ($v_y < -5$)
6	WIND_UP	Attack preparation (reaction window)
7	RECOVERING	Post-attack cooldown (punishment window)

Classification follows a hybrid approach: the FSM state is checked first, then confirmed with kinematic thresholds. A purely kinematic fallback (speed > 15 units/s) handles cases where the FSM is unreadable.

2.5 Action Execution (ActionExecutor)

Action execution happens through a **virtual input device** registered in Unity's InControl system. The class `SyntheticSoulInputDevice` simulates a virtual controller with 8 buttons whose bindings are injected into the game's `HeroActions` via *reflection*.

```
private void BindAction(Type type, string name, InputControl
    control)
{
    var field = type.GetField(name,
        BindingFlags.NonPublic | BindingFlags.Instance |
    BindingFlags.Public);
    if (field != null)
    {
```

```
        var action = field.GetValue(heroActions) as PlayerAction;
        if (action != null && control != null)
            action.AddBinding(new DeviceBindingSource(control.
    Target));
     }
}
```

Listing 2.3: Binding via Reflection (ActionExecutor.cs)

Each action has a configured duration: directional movements persist for a defined interval, while attack, dash and jump are released automatically after a short tap. Before execution, the system verifies private `HeroController` methods (`CanDash()`, `CanAttack()`, `CanJump()`) via reflection to avoid invalid inputs. The system also supports composite actions such as `JUMP_ATTACK` and `DASH_ATTACK`, essential for advanced combat.

2.6 Victory Management and Episode Lifecycle

The `VictoryTracker` manages the victory sequence and maintains session-level training statistics. In the Hall of Gods the Mantis Lords fight requires killing all three mantises; the tracker confirms victory only when `mantisLordsKilled >= 3`.

The episode lifecycle is managed entirely by the mod:

1. **Scene Load:** the scene `GG_Mantis_Lords` is loaded; the mod resets all counters, trackers and detectors.
2. **Hero Restoration:** after loading, the mod waits for the boss intro, restores player HP/Soul and removes residual death objects.
3. **Training Loop:** every 50 ms the mod extracts the state, sends it to Python and waits for the action.
4. **Termination:** upon death or victory the mod sends the final state with the `isDead`/`bossDefeated` flags and automatically reloads the scene.

Chapter 3

State Preprocessing

The raw JSON state received from the mod is transformed into a normalized numerical vector by `preprocess.py`. All values are normalized to $[-1, +1]$ or $[0, 1]$ to stabilize training. Two preprocessing versions are available.

3.1 Version V1 — 34 Features

The base version extracts 34 features organized into 7 groups:

Table 3.1: State vector composition — V1 (34 features).

Group	#	**Detail**
Player Status	5	HP/20, Soul/100, canDash, canAttack, isGrounded
Player Velocity	2	$v_x/20$, $v_y/20$ clamped to $[-1, 1]$
Terrain	5	5 normalized raycast values $[0, 1]$
Boss Position	4	relativeX, relativeY, distance/50, isFacingBoss
Boss Status	4	$v_x/20$, $v_y/20$, HP/100, mantisKilled/3
Boss Intent	4	One-hot encoding (4 classes: Idle/Dash/Drop/Throw)
Hazards	10	2×5 features (relX, relY, v_x, v_y, dist)

3.2 Version V2 — 51 Features

Version V2 extends V1 with 17 additional features from the `MantisAttackDetector`:

Table 3.2: Additional features in V2 (+17 over V1).

Group	#	**Detail**
Primary Pattern	8	One-hot of primary pattern (8 classes, see Tab. 2.2)
Temporal State	3	isWindingUp, isActive, isRecovering
Primary Velocity	2	$v_x/25$, $v_y/25$ of the primary mantis
Secondary Mantis	2	Secondary pattern/7, isActive secondary
Combat Info	2	activeMantisCount/3, anyMantisAttacking

3.3 Frame Stacking

To provide temporal information to the agents, the system uses **frame stacking**: the last $N = 4$ frames are concatenated, yielding an input dimension of:

$$d_{\text{input}} = d_{\text{features}} \times N = 51 \times 4 = 204 \quad \text{(V2)} \tag{3.1}$$

```python
class FrameStacker:
    """Stack of N consecutive frames. State [F] -> [F x N]."""
    def __init__(self, stack_size, state_dim):
        self.frames = deque(maxlen=stack_size)

    def reset(self, initial_state):
        self.frames.clear()
        for _ in range(self.stack_size):
            self.frames.append(initial_state)
        return np.concatenate(list(self.frames))

    def step(self, state):
        self.frames.append(state)
        return np.concatenate(list(self.frames))
```

Listing 3.1: Frame stacker implementation (multi_train.py)

Chapter 4

AI Solution: Deep Reinforcement Learning

4.1 Action Space

Both agents share a discrete action space with 8 actions:

Table 4.1: Agent action space.

ID	Action	In-Game Effect
0	MOVE_LEFT	Move left
1	MOVE_RIGHT	Move right
2	UP	Look up
3	DOWN	Look down
4	JUMP	Jump (or double jump)
5	ATTACK	Nail strike
6	DASH	Quick dash
7	SPELL	Cast spell (Vengeful Spirit, etc.)

4.2 Experience Management (Dynamic Dataset)

Unlike classical supervised learning where the dataset is static, in SyntheticSoul the dataset is **generated dynamically** by the agent during interaction with the environment.

4.2.1 Transition Structure

Each interaction is stored as a tuple e_t:

$$e_t = (s_t,\ a_t,\ r_t,\ s_{t+1},\ d_t) \tag{4.1}$$

where s_t is the current state (204-dimensional vector with V2 frame stacking), a_t the executed action (integer 0–7), r_t the reward, s_{t+1} the next state and d_t the termination flag.

4.2.2 Replay Buffer (DQN)

For DQN we implemented a circular FIFO buffer with capacity $N = 100{,}000$. This allows the agent to discard experiences generated by immature policies while maintaining sufficient variance to avoid overfitting.

```
class ReplayBuffer:
    def __init__(self, capacity=100000):
```

```
        self.memory = deque(maxlen=capacity)  # auto FIFO

    def push(self, state, action, reward, next_state, done):
        self.memory.append(Transition(state, action, reward,
next_state, done))

    def sample(self, batch_size):
        # Random sampling to break temporal correlation
        return random.sample(self.memory, batch_size)
```

Listing 4.1: Replay buffer implementation (dqn_agent.py)

For the PPO agent the dataset is *ephemeral*: it is accumulated over a full episode, used to compute gradients (4 epochs of mini-batches of 128) and discarded immediately after the update, ensuring strictly *on-policy* learning.

4.3 Deep Q-Network (DQN)

4.3.1 Architecture: Dueling DQN

As implemented in `dqn_net.py`, the agent uses a **Dueling DQN** architecture [4] rather than a plain feed-forward network. This architecture separates the estimation of the state value $V(s)$ from the advantage of each action $A(s, a)$. This is especially useful in Hollow Knight because in many frames (e.g. while walking away from the boss) the chosen action is irrelevant to the state's value.

The final Q-value is reconstructed as:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right) \tag{4.2}$$

The parameters and components of the equation are defined as follows:

- s: The input state vector (dimension $d = 204$).
- a: The specific action being evaluated from the action space $\mathcal{A}$.
- θ: The weights of the shared layers (feature extractor) common to both streams.
- β: The weights specific to the Value stream neural network.
- α: The weights specific to the Advantage stream neural network.
- $|\mathcal{A}|$: The cardinality of the action space (total number of available actions, which is 8).

Mechanism and Identifiability: The term $\frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha)$ represents the mean advantage over all possible actions. Subtracting this mean from the advantage $A(s, a)$ is a crucial operation to solve the *identifiability issue*.

Without this subtraction, the equation $Q = V + A$ would be unidentifiable, meaning that adding a constant to V and subtracting the same constant from A would result in the same Q-values, preventing the network from learning a distinct representation for the state value. By forcing the advantages to have zero mean, we ensure that $V(s)$ approximates the true value of the state, while $A(s, a)$ represents the relative importance of each action centered around zero.

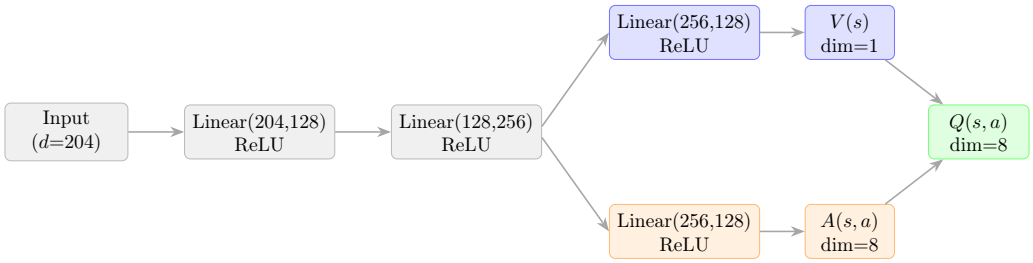

Figure 4.1: Dueling DQN architecture. Shared layers extract features; two separate streams estimate $V(s)$ and $A(s,a)$.

Network Implementation Details

The architecture visualized in Figure 4.1 relies on specific design choices to optimize learning stability and efficiency:

- **Input Layer ($d = 204$):** The input dimension corresponds to the stacked state vector defined in Section 3.3 (51 features $\times$ 4 frames), allowing the network to perceive temporal dynamics such as velocity and acceleration.
- **Hidden Layers Dimensions (128 $\rightarrow$ 256):** The feature extractor follows a compression-expansion structure. The first layer compresses the raw input into 128 units to filter noise, while the second layer expands to 256 units. This expansion increases the network's capacity to model complex non-linear combinations of features before splitting into the two streams. The sizes are powers of 2 to optimize memory alignment and computational efficiency.
- **ReLU Activation:** Each linear transformation is followed by a **Rectified Linear Unit (ReLU)** activation function, defined as $f(x) = \max(0, x)$. ReLU is crucial for introducing non-linearity into the model. Without it, the entire deep network would behave like a single linear regression, incapable of learning complex strategies (conditional logic like "IF health is low AND boss is attacking THEN dodge").

```python
def forward(self, state):
    # Shared layers
    x = F.relu(self.fc1(state))
    x = F.relu(self.fc2(x))
    # Value stream
    value = F.relu(self.value_fc(x))
    value = self.value_out(value)            # [batch, 1]
    # Advantage stream
    advantage = F.relu(self.advantage_fc(x))
    advantage = self.advantage_out(advantage) # [batch, 8]
    # Q = V + (A - mean(A))
    q_values = value + (advantage - advantage.mean(dim=1, keepdim=True))
    return q_values
```

Listing 4.2: Dueling DQN forward pass (dqn_net.py)

4.3.2 DQN Training Mechanisms

To ensure stability and prevent divergence during training, the agent implements several enhancements over the standard DQN algorithm:

- **Double DQN (Decoupled Evaluation):** Standard DQN tends to overestimate action values because the same network is used to both select and evaluate the next action. Double DQN solves this by decoupling the process into two steps (Eq. 4.3):
  1. **Selection:** The *Online Network* (θ) decides which action is optimal for the next state: $\text{argmax}_{a'} Q_\theta(s', a')$.
  2. **Evaluation:** The *Target Network* (θ^-) calculates the value of that specific action.

$$y = r + \gamma \cdot Q_{\theta^-}\left(s', \text{argmax}_{a'} Q_\theta(s', a')\right) \tag{4.3}$$

  This mechanism ensures that if the Online network erroneously overestimates a specific action due to noise, the Target network (which has older weights) acts as a filter, providing a more conservative value estimate.

- **Experience Replay:** A circular buffer ($N = 100,000$) stores transitions to break temporal correlations. Training occurs on random mini-batches of 256 samples.

- **Soft Target Update:** The target network weights update slowly towards the policy weights to maintain stability (with $\tau = 0.005$):

$$\theta^- \leftarrow \tau \cdot \theta + (1 - \tau) \cdot \theta^- \tag{4.4}$$

- **Numerical Robustness:** The agent minimizes the error between the predicted Q-values and the target Q-values using the *Smooth L1 Loss* (Huber Loss). This loss function is less sensitive to outliers compared to Mean Squared Error (MSE), preventing exploding gradients when the error is large. The optimization objective for a batch B is:

$$L_{DQN}(\theta) = \frac{1}{|B|} \sum_{i \in B} \text{SmoothL1}\left(y_i - Q_\theta(s_i, a_i)\right) \tag{4.5}$$

  where y_i is the target value (Eq. 4.3) and the Smooth L1 function is defined as:

$$\text{SmoothL1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases} \tag{4.6}$$

  and apply **Gradient Clipping** ($||\nabla||_{\max} = 1.0$) to prevent exploding gradients.

- **Epsilon-greedy with decay:** Exploration is managed via an exponential decay schedule:

$$\epsilon = \epsilon_{\text{end}} + (\epsilon_{\text{start}} - \epsilon_{\text{end}}) \cdot e^{-\text{steps}/\text{decay}} \tag{4.7}$$

4.4 Proximal Policy Optimization (PPO)

While DQN estimates action values (value-based), PPO directly learns the policy $\pi_\theta(a|s)$ (policy-based). We chose PPO [2] for its stability and ability to converge in complex environments.

4.4.1 Actor-Critic Architecture and Frame Stacking

One of the main challenges in *Hollow Knight* is partial non-Markovianity. While initially designed with an LSTM layer, we transitioned the PPO agent to use the same **Frame Stacking** approach ($N = 4$ frames, yielding a 204-dimensional input) as the DQN agent. This choice significantly improved training stability and aligned the observation space across both algorithms.

Additionally, to mitigate the issue of sparse rewards in continuous environments, the PPO agent utilizes a specialized **Kill Buffer**. This buffer stores complete trajectories of successful episodes (where mantises are killed) and performs extra replay updates on them, providing a consistent gradient towards winning behaviors even during a streak of unsuccessful episodes.

We implemented an *Actor-Critic* network. Since the environment is partially non-Markovian, we overcome the lack of temporal information (e.g., to track projectile trajectories) by augmenting the input with **Frame Stacking**, concatenating the last 4 frames just like in the DQN implementation. The architecture processes the stacked input as follows:

1. **Feature Extractor:** A dense network (two linear layers of 384 units) with *Layer Normalization* processes the stacked state vector to extract spatial and temporal features.
2. **Dual Heads:** The output of the feature extractor splits into two separate heads:
   - **Actor Head (π_θ):** A dense sub-network (128 units) that outputs a probability distribution over the 8 discrete actions (Policy).
   - **Critic Head (V_ϕ):** A dense sub-network (128 units) that estimates the scalar value of the current state (Value Function).

To ensure training stability, we apply *Orthogonal Initialization* to the network weights, which guarantees a healthy gradient flow at the start of the training.

4.4.2 Clipped Objective Function

Standard Policy Gradient methods often suffer from dangerously large updates: if the new policy deviates too much from the old one, performance can collapse irreversibly.

PPO imposes a constraint on the update size using a *Clipped Surrogate Objective*. This ensures the new policy π_θ stays close to the old policy $\pi_{\theta_{old}}$ within a safe "trust region".

The objective function maximized by the agent is:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{4.8}$$

Where the components are defined as follows:

- **Probability Ratio (r_t):** Measures the divergence between the new and old policy:
$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$
- **Advantage ($\hat{A}_t$):** Estimates how much better the chosen action was compared to the average.
- **Clipping ($\epsilon = 0.2$):** This effectively prevents the agent from making updates larger than 20% in a single step, guaranteeing stability.

Total Objective Function. While the clipped objective L^{CLIP} defines the policy update, the full objective maximized by the agent includes the value estimation error and an entropy bonus. The total objective $J(\theta)$ maximized at each update step is:

$$J(\theta) = L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \tag{4.9}$$

The components correspond directly to the implementation in our `PPOAgent`:

- $L^{CLIP}(\theta)$: The clipped surrogate objective that maximizes rewards while limiting policy updates.
- $L^{VF}(\theta)$: The **Smooth L1 Loss (Huber Loss)** for the Critic, replacing the standard Mean Squared Error (MSE) to prevent exploding gradients when the value estimation error is large. Since we maximize J, this term is subtracted (weighted by $c_1 = 0.5$).
- $S[\pi_\theta](s_t)$: The entropy of the policy distribution. We add it (weighted by c_2) to maximize exploration and prevent premature convergence.

4.4.3 Advanced PPO Mechanisms

- **Learning Rate Scheduling:** linear decay from lr_{start} to $\text{lr}_{\text{start}} \times$ lr_end_factor.
- **Dynamic Entropy:** the coefficient c_2 follows a cosine annealing schedule:
$$c_2(t) = c_{2,\text{end}} + (c_{2,\text{start}} - c_{2,\text{end}}) \cdot \frac{1 + \cos(\pi \cdot t/T)}{2} \tag{4.10}$$

4.5 DQN vs PPO Comparison

Although both agents share the same state extraction, action space, and temporal memory mechanism (Frame Stacking), they rely on fundamentally different learning paradigms. This divergence dictated distinct architectural choices and, most importantly, completely different reward shaping strategies.

- **DQN (Off-Policy):** Learns from individual transitions stored in a large Replay Buffer. It bootstraps Q-values via the Bellman equation, which allows it to efficiently propagate **sparse, event-driven rewards** (e.g., massive discrete

bonuses for kills or penalties for taking damage) backward through the buffer. Continuous shaping was avoided to prevent the Q-values from becoming noisy.

- **PPO (On-Policy):** Collects rollouts and computes advantages via Generalized Advantage Estimation (GAE). To prevent high variance in advantage estimates, PPO required a **dense reward shaping** environment, providing small continuous signals at every step (e.g., tracking distance to the boss and movement quality) alongside a specialized **Kill Buffer** to replay successful trajectories.

Table 4.2 summarizes the key differences in their configuration.

Table 4.2: Comparison of the two agents' key characteristics.

Aspect	**DQN**	**PPO**
Paradigm	Value-based, off-policy	Policy-based, on-policy
Network	Dueling DQN (128-256-128)	Actor-Critic (384)
Exploration	Decaying ϵ-greedy	Policy entropy (Cosine decay)
Temporal memory	Frame stacking (4×)	Frame stacking (4×)
Experience	Replay Buffer (100k transitions)	On-policy rollouts + Kill Buffer
Reward Shaping	Sparse (Event-driven)	Dense (Continuous signals)
Target Network	Yes (soft update τ=0.005)	No
Update timing	Every step (if buffer ready)	End of rollout (4 epochs)

4.5.1 Architectural and Memory Comparison

The training process of the two agents differs fundamentally due to their policy learning paradigms:

- **DQN (Off-policy):** Utilizes a Replay Buffer of 100,000 transitions. This allows the agent to learn from past experiences (even those generated by older versions of the policy), making it highly sample-efficient. It reuses data to iteratively update the Q-values towards the optimal Bellman equation.
- **PPO (On-policy):** Data is ephemeral. It collects trajectories over a fixed rollout (e.g., 256 steps), uses them for a gradient update (via GAE and the clipped surrogate objective), and then completely discards them. This guarantees greater training stability, as the model never attempts to learn from actions that its current policy distribution would no longer take.

4.5.2 Complexity Analysis

The architectural choices directly impact the computational footprint of the agents. While both process the same 204-dimensional stacked state:

- **DQN** is highly lightweight, comprising approximately **126k parameters**. The shared feature extractor (128 $\rightarrow$ 256) and the dual streams (128 units each) are optimized for fast inference and efficient Q-value propagation.

- **PPO** is nearly three times larger, containing approximately **328k parameters**. The feature extractor relies on wider layers (384 units) coupled with Layer Normalization, branching into the dual Actor-Critic heads. This increased capacity is necessary to simultaneously approximate both the value function and the complex probability distribution over the discrete action space.

4.5.3 Training Performance Evaluation

To evaluate the relative effectiveness of these approaches, the project includes a visualization suite (`generate_plots.py`) that parses the training logs. By plotting the **Mean Reward** and **Win Rate** across the 4 phases of the Curriculum Learning, we can visually assess the trade-offs: PPO's steady, monotonic improvement driven by on-policy stability versus DQN's potentially faster but noisier convergence driven by experience replay.

Chapter 5

Experiments: 4-Phase Curriculum Learning

Hollow Knight's intrinsic difficulty is extremely high: if the agent tries to attack immediately it dies within seconds without receiving meaningful feedback. To mitigate the *sparse reward* problem and accelerate convergence, we adopted a **4-phase Curriculum Learning** approach.

5.1 Phase Pipeline

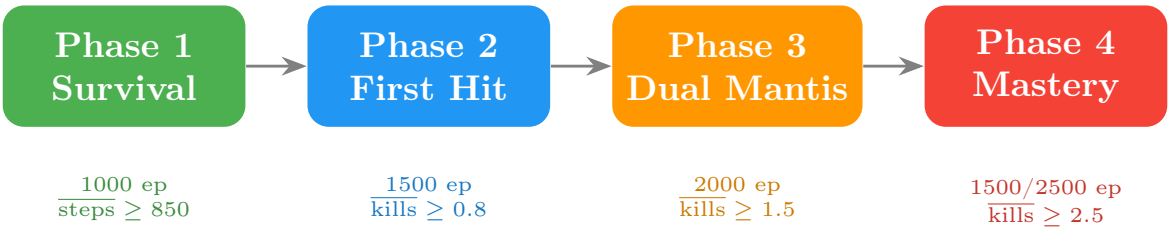

Figure 5.1: Curriculum learning pipeline with automatic promotion conditions.

5.2 Reward Function Details

Each phase has a dedicated reward function implemented in the respective environment wrappers (`env_dqn.py` and `env_ppo.py`), reflecting the different reward shaping strategies. The values are scaled to stabilize training, prioritizing relative importance over absolute magnitude.

Phase 1: Survival

Goal: learn to move, use dash and jump to dodge attacks.

Living reward (per step)	Small incentive (+)
Damage taken	High penalty (−)
Spike damage (additional)	High penalty (−)
Successful dodge (boss attacks, no hit)	Moderate bonus (+)
Wind-up reaction (jump/dash)	Small bonus (+)
Death	Critical penalty (—)
Survival milestone	Periodic bonus (+)

Promotion: based on average survival duration (steps).

Phase 2: First Hit

Goal: learn to strike the boss during recovery windows.

Living reward	Minimal incentive (+)
Damage dealt to boss	Proportional to damage (+)
Hit during recovery (bonus)	Strategic bonus (++)
Attack during wind-up (penalty)	Risk penalty (-)
Damage taken	High penalty (–)
Death	Critical penalty (—)

Promotion: based on average damage dealt per episode.

Phase 3: Dual Mantis

Goal: manage two simultaneous mantises (boss fight phase 2). Introduces **Progressive Kill Rewards** (boosting the reward for the 2nd and 3rd kills) and **HP Threshold Bonuses** to provide an intermediate signal during the dual phase. The damage taken penalty is slightly scaled down to encourage aggression over mere survival.

Phase 4: Mastery

Goal: complete victory, optimize time and no-hit. Combines the mechanics of Phase 3 with massive terminal bonuses for victory. It introduces a **Time Bonus** for fast completions and a **Perfect/Near-Perfect Victory Bonus** for taking zero or minimal damage.

5.3 Pattern Reward Bonus

In addition to the phase reward, a heuristic bonus based on attack patterns is computed to accelerate learning:

Table 5.1: Heuristic reward adjustments based on enemy states.

Mantis Phase	Agent Action	Feedback Effect
Wind-up	Dodge/Move	Reaction Bonus (+)
Wind-up	Attack	Unsafe Penalty (-)
Active attack	Dodge	Evasion Bonus (+)
Recovery	Attack	Punishment Bonus (++)
Secondary active	Dodge	Multi-dodge Bonus (+)

Chapter 6

Multi-Instance Training

6.1 Distributed Training Architecture

The system supports parallel training on **multiple game instances**, each connected to a separate agent instance on a different TCP port. The orchestrator `multi_train.py` manages:

- Launch of N parallel processes (via `multiprocessing` with `spawn` for CUDA compatibility).
- Sharing of best models through a file-locked **Hall of Fame**.
- Periodic synchronization: under-performing instances load models from the Hall of Fame.
- Sequential pipeline through all 4 phases, with automatic best-model transfer to the next phase.

6.2 Hall of Fame

The Hall of Fame maintains the **Top-K** models ($K = 3$) shared among all instances:

- Each instance may occupy at most **one slot** (to ensure diversity).
- Updating a slot requires a minimum improvement of +0.5 reward.
- Ranking is by descending reward; weaker models are discarded.
- Access is protected by **file locking** (`filelock`) for multi-process safety.

Periodically (every `sync_interval` $\times 3$ episodes), instances whose average performance falls below 30% of the global best load a random model from the Hall of Fame.

6.3 Champion System

To systematically track the absolute best model across the entire curriculum, we implemented a **Champion System**. At the end of each training phase run, the orchestrator:

1. Compares the best local models of all active instances against the models stored in the Hall of Fame.
2. Elects the highest-rewarding model as the "Phase Champion".
3. Compares the new champion against any historically saved champion for that phase, overwriting it only if a minimum improvement is met.

This ensures that the final evaluation script (`play.py`) always has access to the most globally optimized weights for each phase.

6.4 Per-Phase Hyperparameter Configuration

A key component of our strategy is the dynamic adjustment of hyperparameters across phases. As the agent progresses from survival to mastery, the learning dynamics must shift from exploration to exploitation and from short-term reactivity to long-term planning.

Table 6.1 details the specific configuration used for each curriculum phase.

Table 6.1: Hyperparameter evolution across the 4 curriculum phases.

Parameter	**P1**	**P2**	**P3**	**P4**
Configuration				
Episodes	1000	1500	2000	1500
Gamma (γ)	0.99	0.995	0.995	0.998
PPO Specific				
Learning Rate	$3{\cdot}10^{-4}$	$1.5{\cdot}10^{-4}$	$5{\cdot}10^{-5}$	$3{\cdot}10^{-5}$
Entropy (ϵ_{ent})	$.08 \rightarrow .03$	$.05 \rightarrow .005$	$.03 \rightarrow .005$	$.02 \rightarrow .003$
DQN Specific				
Learning Rate	10^{-4}	$5{\cdot}10^{-5}$	$3{\cdot}10^{-5}$	$2{\cdot}10^{-5}$
Epsilon ($\epsilon_{start} \rightarrow \epsilon_{end}$)	$.60 \rightarrow .10$	$.40 \rightarrow .08$	$.30 \rightarrow .05$	$.25 \rightarrow .05$

The trends in Table 6.1 reflect three specific adaptation strategies:

1. **Increasing Horizon (γ):** In early phases, survival is immediate, so a lower γ (0.99) suffices. In Phase 4, the agent must optimize the entire fight duration, requiring a higher γ (0.995) to adequately value future rewards.
2. **Decreasing Plasticity (LR):** We start with a high learning rate to quickly learn basic movement. As the policy refines, the LR is annealed to $5 \cdot 10^{-5}$ to perform fine-tuning without destabilizing the learned behaviors.
3. **Reduced Exploration (Entropy/ϵ):** Initial phases require high entropy/epsilon to discover mechanics. In later phases, consistent execution becomes priority, so forced exploration is minimized.

Chapter 7

Implementation Details

7.1 Numerical Stability

To guarantee convergence in a continuous environment like Hollow Knight, several measures were adopted to prevent policy collapse:

- **Layer Normalization:** Applied in the PPO feature extractor to normalize activations, stabilizing the learning process across the frame-stacked inputs.
- **Gradient Clipping:** Caps the gradient norm (0.5 for PPO, 1.0 for DQN) to prevent exploding gradients during backpropagation.
- **NaN Protection:** Detects and zeroes out `NaN` or `Inf` values in the network output to prevent weight corruption.
- **Advantage Normalization:** Normalizes advantages to zero mean and unit variance, decoupling the update magnitude from absolute reward scaling.
- **Smooth L1 Loss (Huber):** Replaces MSE to reduce sensitivity to outliers, preventing destabilization from large error spikes.
- **Weight Initialization:** Uses *Orthogonal* (PPO) and *Xavier* (DQN) schemes to ensure healthy gradient flow at the start of training.

7.2 Technology Stack

Table 7.1: Technologies used in the project.

Component	Technology
Game Mod	C# / Unity, Hollow Knight Modding API, InControl, PlayMaker
Communication	TCP Sockets, JSON (Newtonsoft.Json), blocking reads
RL Framework	Python 3, PyTorch (with CUDA support)
Parallelism	Python multiprocessing (spawn), filelock for synchronization
Logging	Shared CSV (file-locked), custom DesktopLogger to file

7.3 Project Structure

```
SyntheticSoul/
+-- HollowKnightMod/                # C# Mod (Unity)
|   +-- SyntheticSoulMod.cs         # Entry point, lifecycle
|   +-- GameStateExtractor.cs       # Game state extraction
|   +-- ActionExecutor.cs           # Virtual input device
|   +-- SocketCommunicator.cs       # TCP communication
|   +-- MantisAttackDetector.cs     # Pattern detection
```

```
|   +-- VictoryTracker.cs           # Victory tracking
|   +-- DesktopLogger.cs            # Debug logging
+-- AI_Agents/                      # Python RL Framework
    +-- scripts/
    |   +-- train_dqn.py            # Multi-instance orchestrator for DQN
    |   +-- train_ppo.py            # Multi-instance orchestrator for PPO
    |   +-- play.py                 # Evaluation and testing script
    |   +-- preprocess.py           # Preprocessing V1/V2
    +-- src/
        +-- agents/
        |   +-- ppo_agent.py        # PPO Agent
        |   +-- dqn_agent.py        # DQN with Dueling
        +-- models/
        |   +-- actor_critic.py     # Actor-Critic network
        |   +-- dqn_net.py          # DQN/Dueling architectures
        +-- env/
        |   +-- env_dqn.py          # Environment wrapper (Sparse rewards)
        |   +-- env_ppo.py          # Environment wrapper (Dense rewards)
        +-- utils/
            +-- generate_plots.py   # Training metrics plotting
```

Chapter 8

Conclusions and Future Work

The SyntheticSoul project demonstrates the feasibility of applying Deep Reinforcement Learning techniques to a complex boss fight in a commercial game, addressing significant challenges in real-time interfacing, synchronization and reward engineering.

8.1 Main Contributions

- A **complete framework** for training RL agents on Hollow Knight, reusable for other boss fights.
- An **attack pattern detector** based on the combination of FSM reading, kinematic analysis and temporal tracking.
- A **4-phase curriculum learning** system with automatic promotion based on performance metrics.
- A **multi-instance training** infrastructure with a shared Hall of Fame to accelerate convergence.
- A **direct comparison** between DQN (value-based) and PPO (policy-based) on the same task, highlighting the trade-offs between the two approaches.

8.2 Future Work

- Extension to other Hollow Knight bosses (Hornet, Soul Master, Nightmare King Grimm).
- Integration of **visual observations** (pixels) via CNN, eliminating the dependency on direct game-state reading.
- Implementation of **self-play** or **population-based training** for greater strategy diversification.
- Use of **imitation learning** from human replays as pre-training to accelerate initial learning.
- Training speed optimization through game acceleration (`TimeScale > 1`).

Bibliography

[1] Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

[2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347.

[3] Mnih, V., et al. (2015). *Human-level control through deep reinforcement learning*. Nature, 518(7540), 529–533.

[4] Wang, Z., Schaul, T., Hessel, M., et al. (2016). *Dueling Network Architectures for Deep Reinforcement Learning*. Proceedings of ICML.

[5] Hollow Knight Modding API. https://hk-modding.github.io/api/